PumpRadar

Build Guide — Compressed Edition

Leakage-audited, lead-time-aware early warning for cryptocurrency pump-and-dump schemes. A step-by-step, GitHub-based plan for a three-person team to deliver the research paper and a working demo by Thursday 29 October 2026.

Team	Kaashvi Das · Hannah Abraham Ooken · Aveek Bandhyopadhyay
Course	Fundamentals of Machine Learning · Faculty: Mamatha Balachandra
Institution	Mathematics and Computing, Manipal Institute of Technology
Build window	Monday 26 October, 16:00 → Thursday 29 October, demo
Version	26 October 2026 · v2 (React + FastAPI platform) · supersedes the three-week plan

How to read this guide

Section 1 explains what changed. Sections 2–3 fix what you are building and who builds what. Section 4 sets up GitHub in the first hour. Section 5 is the step list, split by person; every step has **Do**, **Why** and **Done when**. Section 6 is the hour-by-hour timeline. Read Sections 1–4 together at the 16:00 kickoff; afterwards each person works from their own part of Section 5.

1. What changed, and why

The original guide planned three weeks. The team has three and a half days and has not started implementation, so the plan is compressed to what three people can build, test and write up by Thursday. **The research question, the core novelty and the classical models are unchanged.** What is cut is infrastructure and polish that the paper and demo do not depend on.

Area	Kept (core of the paper)	Simplified or cut for Thursday
Data	Sapienza Binance events 2018–21; Sapienza labelled features; 1-minute Binance klines	No trade-level (aggTrades) download; no 2021–22 extension; no Telegram listener
Novelty	Leakage audit (inflation per lead time); lead-time curve from market onset; cross-sectional "which coin" scoring; alerts at a stated false-alarm budget	Full discrete-time hazard model becomes a stretch goal; per-tier conformal becomes one quantile threshold
Models	Logistic Regression, KNN, Decision Tree, Random Forest, SVM; LightGBM as reference	Hyperparameter search limited to 2–3 values each
Evaluation	Random split (prior protocol) vs event-grouped chronological split; PR-AUC; Hit@k; event-level bootstrap	Leave-coin-out and external-era tests moved to Future Work
Platform	React + Tailwind frontend and FastAPI backend, deployed on a public URL: live radar, coin view, replay of held-out pumps, honesty and method pages	Minute-by-minute REST polling instead of a WebSocket feed; SQLite instead of a database server; no user accounts
Process	GitHub repo, issues, pull requests, CI, Docker Compose deployment	No MLflow, no container registry, no Kubernetes

> **How a full React + FastAPI product fits in three days**
>
> The platform is the most work-heavy part of the plan, so it is split between two people and built contract-first. Aveek owns the API layer, the frontend and deployment; Hannah owns the scoring engine, because it wraps her feature module. The API response shapes and a mock mode are fixed on Monday evening, so the frontend is built against realistic fake data while the model is still being trained. The site is deployed in mock mode on Wednesday morning and switched to the real model that afternoon. If anything slips, the cut list in Section 6 removes polish, never Replay or the honest evaluation.

2. What you are building

2.1 The question

A pump-and-dump is a coordinated purchase of a small, illiquid coin, announced to followers at a set time, after which the organisers sell to latecomers. Prior work mostly detects a pump once it has started. This project asks: **how many minutes before a pump can we honestly tell it is coming, and on which coin?**

2.2 Four contributions (the novelty)

#	Contribution	What it means in plain words	Gap it fills
1	**Leakage audit**	Score the same models twice: with the random split earlier papers used, and with an honest split (whole pumps in train or test, past trains, future tests, 24 h gap). The difference is the inflation.	Prior early-warning results (Fantazzini and Xiao 2023) used ungrouped cross-validation over near-identical 30-second chunks.
2	**Lead-time curve**	Measure warning quality at 1, 3, 5, 10, 30 and 60 minutes before the pump, with the clock started at the market onset, not the Telegram post.	Prior work reports one 60-minute block or a few seconds-level points.
3	**Cross-sectional scoring**	Every minute, score ~60 pump-prone coins together; the pumped coin is the positive, the others at the same minute are negatives. Report whether it ranks in the top 5.	Detection papers study one coin; target-coin papers use hourly or Telegram data.
4	**Alert budget**	Set the alert threshold so the system raises at most a stated number of false alarms per hour, and show it running live with its own error rate.	Prior work reports AUC, not an operating rule a user can trust.

How to state it in the paper. Claim the combination, not "the first early warning". Earlier papers do predict pumps before announcement; none measures how much of that performance survives honest evaluation, minute by minute, across coins, at a controlled false-alarm rate.

2.3 Why this has not been done

- The public dataset ships features only for the pumped coin, so most follow-up work inherited a single-coin framing.
- At onset the signal is so strong that random and honest splits agree; the inflation only appears minutes earlier, where the one early-warning paper reused the detection protocol.
- Detection researchers and target-coin researchers work at different resolutions with different data and rarely combine methods.

2.4 What exists on Thursday

Deliverable	Contents	Owner
Research paper (6 pages, IEEE)	Experiments A and B, inflation table, lead-time curve, Hit@5, alert-budget results, feature importance	All
Deployed web platform (public URL)	React + Tailwind frontend on Cloudflare Pages; FastAPI backend on a free VM; Radar · Coin view · Replay · Honesty · Method	Aveek, Hannah
GitHub repository	Reproducible code, README, results JSON, figures; data shared through a release	All
Backup video (2–3 min)	Screen recording of the demo, in case the network fails	Aveek

3. Team division of work

One owner per workstream. Each owner can work without waiting for the others, because the hand-offs are fixed files with agreed columns (Section 3.2). Everyone also owns sections of the paper.

	Kaashvi — Models and evaluation	Hannah — Data and features	Aveek — Platform and repository
Owns	Experiment A (leakage audit), Experiment B (cross-sectional models), metrics, thresholds, figures of results	Event table, Binance downloads, onset labels, cross-sectional dataset, feature module and its test; the platform's live scoring engine and replay files	GitHub setup and CI; FastAPI API layer and contract; React frontend (5 pages); deployment; demo and video
Produces	metrics_A.json, metrics_B.json, model.joblib, threshold.json, result figures	events.parquet, xsec.parquet, features.py, api/engine/, replay/*.json, data figures	api/ (routes, schemas), web/, deploy/, README, demo video
Paper	Results (Method drafted by Hannah)	Data, Related Work, Method draft	System, Introduction
Shared	Abstract, Discussion and Limitations, final read	Abstract, Discussion, number check	Abstract, Discussion, demo rehearsal

3.2 Hand-off contracts

These files are the only interfaces between people. Agree the columns at kickoff; anyone changing them opens a pull request and tags the consumer.

File	Producer → consumer	Contents	Due
data/processed/events.parquet	Hannah → Kaashvi, Aveek	event_id, pair, t_telegram, t_onset, era (train / test)	Mon 21:00
data/processed/sapA.parquet	Kaashvi → Kaashvi	Sapienza 25 s features + pump_index, date, lead_min, y_1…y_60	Mon 22:00
src/pumpradar/features.py	Hannah → Kaashvi, Aveek	compute_features(bars) → one row per (pair, minute) with all model features	Tue 13:00
data/processed/xsec.parquet	Hannah → Kaashvi, Aveek	event_id, pair, minute, lead_min, era, features…	Tue 13:00
api/schemas.py → web/src/api/types.ts	Aveek → Hannah, Aveek	Pydantic response models for every endpoint; TypeScript types generated from them	Mon 20:00
models/model.joblib, threshold.json, features.json	Kaashvi → Hannah, Aveek	Frozen models (dictionary: horizon → model), alert threshold, ordered feature list	Wed 13:00
replay/*.json	Hannah → Aveek	Minute-by-minute risk for every coin in each test-era event	Wed 15:30
results/*.json, figures/*.png	Kaashvi, Hannah → All	Every number and figure used in the paper	Wed 18:00

4. GitHub collaboration setup (first hour)

Do G1–G6 together on Monday at 16:00. After that, every change reaches `main` through a pull request.

G1 Create the repository and add the team · Aveek

Do

1. Create a private repo `kaashvidas/pumpradar`: `gh repo create kaashvidas/pumpradar --private --clone`.
2. Add Hannah and Aveek as collaborators (Settings → Collaborators, role Write).
3. Protect `main`: require a pull request with 1 approval and passing CI. Allow the author to merge after approval.

Why. Nobody overwrites anyone else's work; main always runs.

Done when. All three have cloned the repo and can open a pull request.

G2 Create the folder layout and starter files · Aveek

Do. Commit the layout below, an empty `README.md`, `requirements.txt`, `.gitignore` and `.github/CODEOWNERS`.

Why. Folder ownership stops two people editing the same file; ignoring data stops multi-gigabyte commits.

Done when. The layout is on main.

```
pumpradar/
├── data/              # never committed (in .gitignore); raw/ and processed/
├── src/pumpradar/
│   ├── events.py      # Hannah   — event table and onset
│   ├── download.py    # Hannah   — Binance archive downloads
│   ├── features.py    # Hannah   — compute_features(), shared by research and app
│   ├── splits.py      # Kaashvi  — random vs honest splits
│   ├── metrics.py     # Kaashvi  — PR-AUC, Hit@k, budget threshold, bootstrap
│   ├── exp_a.py       # Kaashvi  — Experiment A (leakage audit)
│   └── exp_b.py       # Kaashvi  — Experiment B (cross-sectional)
├── api/               # Aveek    — FastAPI app: main.py, schemas.py, routes/
│   └── engine/        # Hannah   — ingest.py (Binance polling), score.py (features + model)
├── web/               # Aveek    — React + Vite + Tailwind frontend
├── deploy/            # Aveek    — docker-compose.yml, Caddyfile
├── replay/            # Hannah   — precomputed replay JSON (shared via the release)
├── models/            # frozen model files (large files go to a release)
├── results/           # metrics JSON (committed) and figures/
├── tests/             # test_features.py (future-shift test)
├── notebooks/         # exploration only, named <initials>_<topic>.ipynb
└── paper/             # LaTeX source or Overleaf link
```

```
# .gitignore
data/
*.zip
*.parquet
models/*.joblib
__pycache__/
.venv/
.ipynb_checkpoints/
```

```
# .github/CODEOWNERS  (replace with real GitHub usernames)
src/pumpradar/events.py     @hannah-username
src/pumpradar/download.py   @hannah-username
src/pumpradar/features.py   @hannah-username
src/pumpradar/splits.py     @kaashvidas
src/pumpradar/metrics.py    @kaashvidas
src/pumpradar/exp_*.py      @kaashvidas
api/                        @aveek-username
api/engine/                 @hannah-username
web/                        @aveek-username
deploy/                     @aveek-username
```

```
# requirements.txt (one package per line in the real file)
pandas  numpy  pyarrow  scikit-learn  lightgbm  shap  matplotlib  requests  joblib
fastapi  uvicorn[standard]  sse-starlette  httpx  pydantic-settings  pytest  nbstripout
# web/ uses Node 20+ and package-lock.json (npm ci)
```

G3 Labels, milestones and issues · Aveek

Do

1. Labels: `data`, `model`, `api`, `web`, `paper`, `blocker`.
2. Milestones: Mon 26, Tue 27, Wed 28, Thu 29.
3. One issue per step in Section 5 (D1…D6, M1…M8, B1…B5, F1…F7, W1…W3), assigned and placed in its milestone. Example: `gh issue create -t "D1 Build event table" -l data -m "Mon 26" -a @me`.
4. Create a GitHub Project board with columns Todo · Doing · Review · Done and add all issues.

Why. The board is the stand-up: anyone can see what is blocked without asking.

Done when. Every step in Section 5 has an issue on the board.

G4 Local setup · Each person

Do. `python -m venv .venv`, activate it, `pip install -r requirements.txt`, then `nbstripout --install`. Aveek also installs Node 20+ and runs `npm ci` in `web/`.

Why. Identical libraries on three laptops; notebooks without outputs never cause merge conflicts.

Done when. `python -c "import sklearn, lightgbm, fastapi"` works for everyone.

G5 Minimal CI · Aveek

Do. Add `.github/workflows/ci.yml` with two jobs on every pull request: Python (`pip install -r requirements.txt && pytest -q`) and web (`cd web && npm ci && npm run build`, which also type-checks).

Why. The feature leakage test (D5) then guards every later change to features.py.

Done when. A pull request shows a green check.

G6 Agree the working rules · All

Do. Read the rules below aloud and agree sync times.

Why. Three people editing one codebase for three days without rules produces merge conflicts at the worst moment.

Done when. Rules pinned in the README.

Daily Git routine

```
git switch main && git pull                 # start every work block from the latest main
git switch -c feat/<area>-<short-name>      # e.g. feat/data-events, feat/model-exp-a, feat/web-replay
# ... work, commit often:  git add -p  &&  git commit -m "data: add onset detection"
git push -u origin feat/<area>-<short-name>
gh pr create --fill --reviewer <teammate>   # link the issue: "Closes #12" in the description
# reviewer: approve or comment within 30 minutes; author merges with "Squash and merge"
```

- **Small pull requests** (under about 300 lines), at least two per person per day.
- **Commit messages** start with the area: `data:`, `model:`, `api:`, `web:`, `paper:`, `fix:`.
- **Data and big model files never enter Git.** Share them as assets of a GitHub release: `gh release create data-v1 data/processed/*.parquet`; teammates fetch with `gh release download data-v1 -D data/processed`.
- **Results come from scripts, not notebooks.** Every number in the paper is written to `results/*.json` by code in `src/`.
- **Syncs:** 15 minutes at 13:00 and 22:00 each day (Section 6). A blocked person adds the `blocker` label and messages the owner immediately.
- **Seed 42** everywhere; **UTC** for every timestamp.

5. Step-by-step build

Steps are ordered by when they start. Each is one GitHub issue. Times are targets; the timeline in Section 6 shows how they interleave.

5A. Data and features — Hannah

D1 Build the event table · Hannah · Mon 17:00–19:00

Do

1. Download `pump_telegram.csv` and the `labeled_features/` folder from github.com/SystemsLab-Sapienza/pump-and-dump-dataset.
2. Keep rows with exchange = Binance. Form the pair as symbol + "BTC" (e.g. VIB → VIBBTC). Combine date and hour into a UTC timestamp.
3. Treat the same pair within 30 minutes as one event. Assign `event_id`.
4. Set `era` = train for events up to 31 Dec 2019, test from 1 Jan 2020.
5. Save `events.parquet` and print counts per era.

Why. Every later step keys on this table; the era split is what makes Experiment B forward in time.

Done when. Event count close to the proposal's 520, printed by era; columns match Section 3.2.

D2 Download 1-minute klines (runs overnight) · Hannah · Mon 19:00 start, background overnight

Do

1. Universe = all pairs that appear in the event table (about 85 pump-prone coins). These coins are harder negatives than random coins, which makes the test stricter.
2. For each event date and the day before it, fetch every universe pair's daily file `https://data.binance.vision/data/spot/daily/klines/{PAIR}/1m/{PAIR}-1m-{YYYY-MM-DD}.zip`. Skip 404s (the coin was not listed that day) and log them.
3. Use 16 parallel downloads (`concurrent.futures.ThreadPoolExecutor`) and skip files already on disk, so the script can be re-run.
4. If daily files return 404 for most 2018 dates, switch to monthly files (`.../monthly/klines/{PAIR}/1m/{PAIR}-1m-{YYYY-MM}.zip`) for those months.

Why. Klines are free, need no API key, and carry volume, trade count and taker-buy volume for every coin — enough for all cross-sectional features, and the same fields the live app can fetch.

Done when. Manifest of downloaded and missing files; at least 90% of event-days covered for the pumped coin.

```
COLS = ["open_time","open","high","low","close","volume","close_time","quote_volume",
        "count","taker_buy_base","taker_buy_quote","ignore"]
def load_kline_zip(path):
    df = pd.read_csv(path, header=None, names=COLS)
    if df.open_time.iloc[0] > 1e14:          # some archive files use microseconds
        df["open_time"] //= 1000
    df["minute"] = pd.to_datetime(df.open_time, unit="ms", utc=True)
    return df[["minute","close","high","low","quote_volume","count","taker_buy_quote"]]
```

D3 Find each pump's market onset · Hannah · Mon 20:00–22:00

Do

1. For each event, look at its pair's bars from 5 minutes before to 2 minutes after the Telegram time.
2. Onset = first minute whose taker-buy volume is at least 10 robust standard deviations above that pair's previous 24 hours (excluding the last hour). If none, keep the Telegram minute and flag `no_spike`.
3. Write `t_onset` into events.parquet and plot a histogram of Telegram minus onset.

Why. Organisers often start buying before posting. Measuring lead time from the post would count the pump itself as warning.

Done when. Histogram saved to figures/; number of no_spike events reported.

D4 Feature module and cross-sectional dataset · Hannah · Tue 09:00–13:00

Do

1. In `features.py`, write `compute_features(bars)` producing, for windows of 5, 15 and 60 minutes: robust z-scores of taker-buy volume, total volume, trade count and absolute return; buy share (taker-buy ÷ total volume); and quiet accumulation (volume z minus absolute-return z).
2. Add market-relative features: each z ranked among all universe coins at the same minute (percentile 0–1).
3. Build the sample: for every event, minutes from onset − 120 to onset − 1, all universe coins trading that day. `lead_min` = minutes to onset for the pumped coin, empty for the others.
4. Save `xsec.parquet` and upload it to the `data-v1` release.

Why. Each feature asks "how unusual is this coin now, compared with its own normal and with every other coin" — what manipulation looks like and what coin identity cannot fake. Raw price and coin name are excluded on purpose.

Done when. xsec.parquet in the release; about 120 minutes × ~60 coins per event; no feature more than 2% missing.

```
def robust_z(s: pd.Series, w: int) -> pd.Series:
    roll = s.rolling(w, min_periods=w).sum()
    ref = roll.shift(60)                                  # baseline ends 60 min before t
    q = ref.rolling(1380, min_periods=600)                # previous ~23 hours
    iqr = q.quantile(0.75) - q.quantile(0.25)
    return (roll - q.median()) / (iqr / 1.349 + 1e-9)

# per pair:   df[f"z_buy_{w}"] = df.groupby("pair").taker_buy_quote.transform(lambda s: robust_z(s, w))
# per minute: df[f"r_buy_{w}"] = df.groupby("minute")[f"z_buy_{w}"].rank(pct=True)
```

D5 Leakage unit test · Hannah · Tue 21:00–22:00

Do

1. In `tests/test_features.py`: take one pair's bars, compute features, then multiply every volume after a chosen minute t by 1,000 and compute again.
2. Assert every feature at or before t is unchanged. Loop over every feature column.
3. Assert the model feature list contains no `pair`, `symbol` or price-level column.

Why. This single test guards the paper's central claim: no feature looks into the future or names the coin.

Done when. Test passes in CI.

D6 Data note and figures for the paper · Hannah · Wed 09:00–10:00

Do

1. Write `results/data_summary.json`: events per era, coverage, universe size per event, onset offsets, no_spike count.
2. Produce Figure: onset offset histogram; Table: dataset summary.

Why. Reviewers read the data section first; numbers generated by script stay correct if the data change.

Done when. Both exhibits in figures/ and referenced in the Data section.

5B. Models and evaluation — Kaashvi

M1 Prepare Experiment A data from the Sapienza features · Kaashvi · Mon 17:00–19:30

Do

1. Load the 25-second file from `labeled_features/`; check column names first.
2. Onset per pump = earliest chunk with `gt == 1`. Compute `lead_min` for every chunk.
3. Keep chunks in the 6 hours before onset (lead 0–360 min); drop chunks after onset.
4. Create labels `y_k = lead_min ≤ k` for k = 1, 3, 5, 10, 30, 60.

Why. Experiment A uses the exact features of prior work, so any inflation found cannot be blamed on new features.

Done when. sapA.parquet saved; positives per horizon printed.

```
df = pd.read_csv("labeled_features/features_25S.csv.gz", parse_dates=["date"])   # check df.columns
onset = df[df.gt == 1].groupby("pump_index").date.min().rename("onset")
df = df.join(onset, on="pump_index")
df["lead_min"] = (df.onset - df.date).dt.total_seconds() / 60
df = df[(df.lead_min > 0) & (df.lead_min <= 360)]
for k in [1, 3, 5, 10, 30, 60]:
    df[f"y_{k}"] = (df.lead_min <= k).astype(int)
```

M2 Split and metric functions · Kaashvi · Mon 20:00–23:00

Do

1. **P0 (prior protocol):** rows shuffled at random, 50/50; also 5-fold shuffled KFold.
2. **P2 (honest):** pumps sorted by date; earliest 70% train, latest 30% test; every chunk of a pump on one side; drop training rows within 24 hours of the first test timestamp.
3. Metrics: PR-AUC (`average_precision_score`) per horizon; Hit@k; threshold at an alert budget; event-level bootstrap (resample pumps, 1,000 times).

Why. P0 reproduces what earlier papers did; P2 is how the system would be used. The gap between them is Contribution 1.

Done when. Unit test: no pump_index appears in both train and test under P2; a constant score gives PR-AUC equal to the positive rate.

```
def split_random(df, seed=42):
    idx = df.sample(frac=1, random_state=seed).index
    return idx[: len(idx)//2], idx[len(idx)//2 :]

def split_honest(df, key="pump_index", frac=0.7, purge="24h"):
    first = df.groupby(key).date.min().sort_values()
    train_keys = set(first.index[: int(len(first) * frac)])
    tr = df[df[key].isin(train_keys)]
    te = df[~df[key].isin(train_keys)]
    tr = tr[tr.date < te.date.min() - pd.Timedelta(purge)]
    return tr.index, te.index

def threshold_at_budget(neg_scores, alerts_per_hour, n_coins):
    alpha = alerts_per_hour / (60 * n_coins)          # allowed false alarms per coin-minute
    return np.quantile(neg_scores, 1 - alpha)
```

M3 Run Experiment A: the leakage audit · Kaashvi · Tue 09:00–13:00

Do

1. For each horizon k and each model (LR, KNN, DT, RF, SVM, LightGBM): train and score under P0 and P2.
2. Use class weights (`class_weight="balanced"`); for KNN and SVM train on all positives plus 20,000 sampled negatives.
3. Run twice: with prior work's full feature set, and with a clean set that drops price level and time-of-day.
4. **Canary:** add a one-hot coin identity column. If it lifts PR-AUC under P0 but not P2, the random split rewards recognition.

Why. This is the paper's first and most robust finding: whichever way it falls, it is publishable.

Done when. results/metrics_A.json with PR-AUC per model × horizon × protocol, and inflation = P0 − P2.

M4 Lead-time curve and figures for Experiment A · Kaashvi · Tue 14:00–17:00

Do

1. Plot PR-AUC against lead time (1–60 min) for P0 and P2, best two models, with bootstrap bands.
2. Plot inflation per horizon as bars.

Why. The curve is the headline figure: how fast genuine warning fades, and how much prior protocols hid that.

Done when. figures/leadtime_A.png and figures/inflation_A.png generated by a script.

M5 Run Experiment B: cross-sectional models · Kaashvi · Tue 18:00 – Wed 11:00

Do

1. Load xsec.parquet. Train on era = train, test on era = test; hold out the last 20% of training events for tuning the threshold.
2. For each horizon k, label = pumped coin with lead ≤ k; all other rows are 0. Train the six models with class weights.
3. Compute PR-AUC per horizon and Hit@1 / Hit@5: at minute onset − k, rank all coins in that event and check where the pumped coin lands.
4. Compare with the proposal's sampled 1:5 negatives design, scored on the realistic test set (one table row).

Why. This answers the question a trader faces — which coin, how early — at the true base rate (about 1 in 60).

Done when. results/metrics_B.json with PR-AUC and Hit@k per model and horizon.

M6 Alert threshold and frozen model · Kaashvi · Wed 11:00–13:00

Do

1. Pick the deployed model: best tuning PR-AUC at k = 10 (ties → simpler model).
2. Calibrate the chosen model's scores with isotonic regression fitted on the tuning events (`CalibratedClassifierCV(model, method="isotonic", cv="prefit")`) so a displayed 20% means a pump follows about one time in five; report the Brier score.
3. Set the threshold with `threshold_at_budget` for 2 false alerts per hour across the universe; report precision, recall and median first-alert lead time on the test era at that threshold.
4. Save `model.joblib` (a dictionary: horizon → fitted model), `threshold.json` and `features.json`; upload to the release and tell Hannah and Aveek.

Why. The website shows the score as a confidence level to investors, so it must be a true probability; the threshold then turns it into an alert rule with a promised false-alarm rate.

Done when. Calibrated models and threshold in the release by Wed 13:00.

M7 Interpretability · Kaashvi · Wed 14:00–17:00

Do

1. Logistic regression coefficients per horizon (standardised features).
2. Permutation importance on the test era for the deployed model; SHAP summary if time allows.
3. Heatmap: importance by feature family × horizon (which signals warn early, which only near onset).

Why. Fulfils the proposal's interpretability objective and gives the platform its "top drivers" text.

Done when. figures/importance.png and a short paragraph in results/notes.md.

M8 Results freeze · Kaashvi · Wed 18:00

Do. Re-run both experiments from a clean checkout with one command (`python -m pumpradar.exp_a && python -m pumpradar.exp_b`), commit results/ and tag `results-v1`.

Why. Numbers in the paper must match code anyone can run.

Done when. Tag exists; no result changes after it.

> **Stretch goal (only if M1–M8 finish by Wednesday 14:00)**
>
> Train one logistic regression in hazard form: one row per coin-minute per time slice, with the slice as a feature and label 1 only in the slice where the pump begins. Combine slices into risk for any horizon. Report how often the separate per-horizon classifiers give a higher risk for a shorter window than a longer one (an inconsistency the hazard form cannot produce). Otherwise mention it as Future Work.

5C. Platform — React + FastAPI (Aveek, with Hannah on the scoring engine)

The platform is a real, deployable product: a FastAPI backend that polls Binance every minute and scores every coin, and a React + Tailwind frontend on a public URL. Two rules make it buildable in three days. **Contract first:** the API's response shapes are fixed on Monday and the backend has a mock mode, so the frontend never waits for the model. **One feature module:** the scoring engine imports Hannah's `compute_features`, so the website runs exactly the model in the paper.

What the website delivers

Feature	How it works	Steps
Live prices for every Binance spot coin	One public WebSocket stream (`!miniTicker@arr`) pushes price, 24 h change and volume for all listed coins about once a second; the API relays it to the browser	B2, F2
Pump-risk score with a confidence level	Every minute, each scored coin gets a calibrated probability of a pump starting within 1, 5, 10, 30 and 60 minutes, shown as a percentage and a level: Normal, Watch, Alert	M6, B3, F2
Threshold flag	A coin is flagged when its risk crosses the threshold set for a budget of 2 false alerts per hour; Watch appears at half the threshold	M6, B3
Investor alerts	On-screen toast and a browser notification the moment a coin is flagged; optional Telegram channel that posts every alert with its reasons	F2, B6
Why it was flagged	The five main drivers in plain language, next to the price chart and the risk line	B3, F3
Past-pump demo window	Replay any held-out historical pump minute by minute and see when the alert fired; a scoreboard lists every past event with its warning lead time and rank	B4, F4
Honest record	Real alerts per hour against the budget, and precision, recall and lead time on held-out pumps	F5

Which coins get a risk score

Live **prices** cover every coin listed on Binance spot. Live **risk scores** cover the BTC-quoted, lower-liquidity coins, because that is the market the model was trained and tested on (all 2018–21 pumps were on BTC pairs). Every other coin is shown with its live price and the label "not scored". Scoring USDT pairs would require retraining on USDT data and is listed as Future Work; showing a score the model was never tested on would contradict the paper.

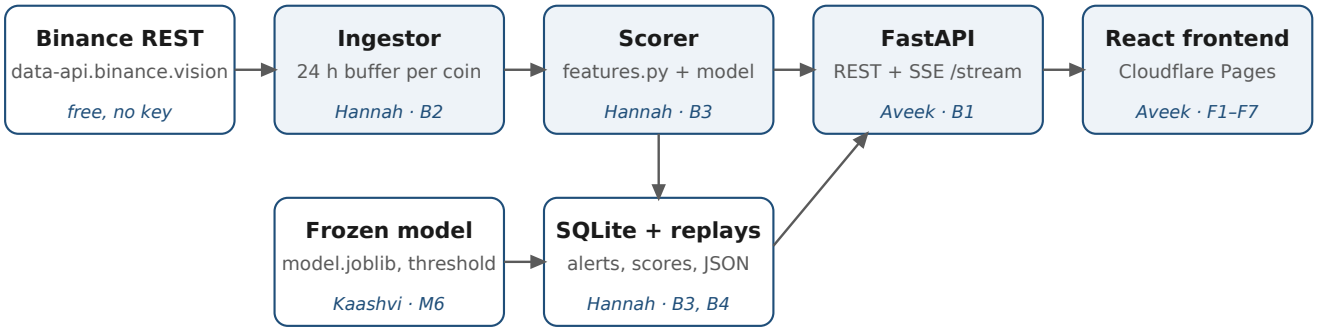


Figure: platform architecture and owners. Every minute the ingestor pulls bars from Binance, the scorer turns them into risk and alerts, and FastAPI pushes them to the browser over HTTPS. Ingestor, scorer and API run in one Docker Compose service on the VM; the frontend is static on Cloudflare Pages.

Tech stack

Layer	Choice	Why
Frontend build	Vite 8, React 19, TypeScript	Fast builds; largest ecosystem; static output hosts anywhere for free
Styling and components	Tailwind CSS v4, shadcn/ui (Radix), Lucide icons	High-end, accessible components you own and restyle; dark mode built in
Data and routing	TanStack Query, TanStack Router, TanStack Table	Cached server state; filters in the URL so any view is shareable; sortable 100-row radar
Charts	TradingView Lightweight Charts v5 (+ vertical-line plugin); Recharts	Real candlesticks with risk overlay and alert markers; small charts for metrics
Motion and feedback	Motion, Sonner toasts, browser notifications	Rows re-rank smoothly each minute; alerts are noticed
API types	openapi-typescript	TypeScript types generated from FastAPI, so front and back cannot drift
Backend	FastAPI, Pydantic v2, Uvicorn (1 worker), sse-starlette, httpx	Async API with automatic OpenAPI; Server-Sent Events push each minute
Model runtime	pumpradar package: features.py, model.joblib, threshold.json	Same code and frozen model as the paper
Storage	SQLite (alerts, scores, health); replay JSON files	Zero setup; enough for one server
Market data	Binance public WebSocket (data-stream.binance.vision, `!miniTicker@arr`) for live prices of all coins; REST (data-api.binance.vision: klines, exchangeInfo) for the 1-minute bars the model scores	Both free and keyless; one WebSocket connection carries every coin's price; ~100 REST requests per minute is far below the limits
Hosting	Backend: free VM in Mumbai, Hyderabad or Singapore (e.g. Oracle Cloud Always Free), Docker Compose + Caddy (HTTPS). Frontend: Cloudflare Pages	Binance is reachable from these regions (it blocks the US); Caddy issues free TLS certificates

Interface design: what "high-end" means here

Element	Specification
Look	Dark-first trading-terminal style (near-black #0B0D10 background, #12151A cards, 1 px #1F242C borders); full light theme as a toggle
Colour meaning	One brand accent (electric blue) for interaction only. Risk uses one ramp: grey → amber → red. Green is never used for "safe", so nobody reads it as a buy signal
Type	Inter for text; JetBrains Mono with tabular figures for every number, so columns do not jitter when values update
Layout	Left sidebar (Radar, Replay, Honesty, Method); top bar with live-status dot, last-update time, horizon selector; 8 px spacing grid; works at 375 px phone width
Motion	Rank changes animate in 250 ms; new alerts slide in with a toast; nothing flashes or auto-plays sound
States	Skeleton loaders, empty states with one sentence of guidance, and a clear banner when the live feed drops ("Reconnecting — last data 14:02 UTC")
Trust cues	Every alert shows its reasons; every page footer says "Research tool. Not financial advice. No trades are placed."

Backend steps

B1 API skeleton, contract and mock mode · Aveek · Mon 17:00–20:00

Do

1. Create `api/` with FastAPI. Define Pydantic response models in `api/schemas.py` for every endpoint below, then commit them — this is the contract.
2. Endpoints: `GET /markets` (every spot coin: symbol, price, 24 h change, volume, scored or not); `GET /radar` (scored coins: pair, tier, risk per horizon, level, alert, sparkline); `GET /coin/{pair}` (bars, risk series, alerts, top 5 drivers); `GET /alerts`; `GET /stream` (SSE: a `prices` event every 2 s and a `tick` with scores and new alerts every minute); `GET /replay/events`; `GET /replay/{event_id}`; `GET /honesty`; `GET /health`.
3. Mock mode (`PUMPRADAR_MOCK=1`): the engine emits plausible random scores for 60 fake coins every 10 s.
4. Generate frontend types: `npx openapi-typescript http://localhost:8000/openapi.json -o web/src/api/types.ts`.

Why. With the contract and mock mode fixed on Monday, the frontend and the scoring engine are built in parallel and meet on Wednesday without surprises.

Done when. `curl -N localhost:8000/stream` prints a tick every 10 s in mock mode; types file committed.

```
# api/engine/prices.py — live prices for every Binance spot coin (no API key)
URL = "wss://data-stream.binance.vision/ws/!miniTicker@arr"
async def price_stream(store: dict):
    while True:
        try:
            async with websockets.connect(URL, ping_interval=20) as ws:
                async for msg in ws:                        # about one message per second
                    for t in json.loads(msg):               # s symbol, c close, o open, q quote volume
                        store[t["s"]] = {"price": float(t["c"]), "open": float(t["o"]),
                                         "vol": float(t["q"])}
        except Exception:
            await asyncio.sleep(5)                          # reconnect; Binance drops connections after 24 h
```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    task = asyncio.create_task(engine.run())        # poll Binance each minute, score, broadcast
    yield
    task.cancel()

app = FastAPI(title="PumpRadar API", lifespan=lifespan)
app.add_middleware(CORSMiddleware, allow_origins=[settings.web_origin], allow_methods=["GET"])

@app.get("/radar", response_model=list[CoinRisk])
def radar():
    return engine.latest()

@app.get("/stream")
async def stream(request: Request):
    async def gen():
        q = engine.subscribe()
        while not await request.is_disconnected():
            tick = await q.get()
            yield {"event": "tick", "data": tick.model_dump_json()}
    return EventSourceResponse(gen())
```

B2 Live ingestor · Hannah · Tue 14:00–17:30

Do

1. In `api/engine/ingest.py`: on start, fetch BTC pairs from `exchangeInfo` (status TRADING), drop the 20 most liquid by 24-hour volume, and backfill 1,500 one-minute bars per pair (two `/klines` calls with `limit=1000` and `endTime`).
2. Every minute, 3 s after the bar closes, fetch the last 3 bars per pair concurrently (httpx, at most 20 requests in flight) and append closed bars to an in-memory buffer (last 1,500 minutes per pair).
3. If a fetch fails, retry once; if a minute is missing, backfill it on the next cycle. Record failures for `/health`.
4. Live prices for every coin: a second task holds one WebSocket connection to `wss://data-stream.binance.vision/ws/!miniTicker@arr`, keeps the latest price, 24 h change and volume per symbol, and the API pushes a `prices` event every 2 s. Reconnect with backoff; Binance closes every connection after 24 hours.

Why. Prices must feel live (seconds), while the model is defined on 1-minute bars, so scores update every minute. One WebSocket covers every coin's price; minute polling of bars is the robust choice for scoring. Neither needs an API key.

Done when. Prices update every 2 s for all coins; the bar buffer holds 24 h for every scored pair and one hour matches a direct REST download.

B3 Scorer, alerts and storage · Hannah · Tue 17:00–21:00

Do

1. In `api/engine/score.py`: after each ingest, run `compute_features` on the buffer, keep each pair's latest row, and predict risk for each horizon with the model dictionary (dummy until Wed 13:00).
2. Level per coin at the 10-minute horizon: **Alert** if risk ≥ threshold, **Watch** if ≥ half the threshold, otherwise Normal. Raise an alert event when a coin enters Alert; at most one per pair per 30 minutes.
3. Top 5 drivers per alert: feature value × importance, turned into a sentence from a template.
4. Write scores (kept 48 h), alerts (kept) and health rows to SQLite; push the tick to `engine.subscribe()` queues.

Why. Scoring all coins together reproduces the cross-sectional design exactly as evaluated in the paper.

Done when. Ticks contain real scores for every live pair within 10 s of each minute.

B4 Replay files · Hannah · Wed 13:30–15:30

Do

1. After Kaashvi freezes the model (Wed 13:00), score every test-era event's rows in xsec.parquet with it.
2. Write `replay/{event_id}.json`: per minute from onset − 120 to onset + 10, every coin's risk, plus the pumped coin's bars; and `replay/index.json` with event, date, coin, first-alert lead and rank.

Why. Precomputed replays load instantly, cost nothing on the server, and use only held-out events.

Done when. Every test event has a file; its first-alert lead matches Kaashvi's offline result for three spot-checked events.

B5 Deployment · Aveek · Wed 09:00–11:00 and 15:30–16:00

Do

1. Create the VM (Ubuntu, India or Singapore region). **First** run `curl https://data-api.binance.vision/api/v3/ping` on it; if refused, change region.
2. Install Docker; copy `deploy/docker-compose.yml` and `Caddyfile`; use the free hostname `api.<VM-IP-with-dashes>.sslip.io` so Caddy can obtain HTTPS.
3. Frontend: connect the repo to Cloudflare Pages (build `npm run build`, output `web/dist`, variable `VITE_API_URL`). Set CORS to the Pages domain.
4. Deploy in mock mode Wednesday morning; redeploy with the real model and replay files at 15:30; add a free uptime monitor on `/health`.

Why. Deploying early, in mock mode, flushes out region, HTTPS and CORS problems while there is still time to fix them.

Done when. Public URL shows live scores from the real model by Wed 16:00.

```
# deploy/docker-compose.yml
services:
  api:
    build: ../api
    env_file: ../.env                    # PUMPRADAR_MOCK=0, WEB_ORIGIN=https://pumpradar.pages.dev
    volumes: ["../models:/app/models:ro", "../replay:/app/replay:ro", "../state:/app/state"]
    command: uvicorn main:app --host 0.0.0.0 --port 8000 --workers 1
    restart: unless-stopped
  caddy:
    image: caddy:2
    ports: ["80:80", "443:443"]
    volumes: ["./Caddyfile:/etc/caddy/Caddyfile", "caddy_data:/data"]
volumes: {caddy_data: {}}

# deploy/Caddyfile
api.<vm-ip-with-dashes>.sslip.io {
    reverse_proxy api:8000
}
```

B6 Telegram alert channel (optional, only if ahead of schedule) · Hannah or Aveek · Wed 16:00–17:00

Do

1. Create a bot with Telegram's @BotFather and a public channel; add the bot as an admin.
2. When B3 raises an alert, call `https://api.telegram.org/bot<TOKEN>/sendMessage` with the coin, risk, horizon and top three reasons. Keep the token in `.env`, never in Git.

Why. Browser notifications only reach people with the site open; a channel reaches anyone who subscribes.

Done when. A mock alert appears in the channel within 10 s.

Frontend steps

F1 Scaffold, design tokens and app shell · Aveek · Mon 20:00–23:00

Do

1. `npm create vite@latest web -- --template react-ts`; add Tailwind v4 (`@tailwindcss/vite`), `npx shadcn@latest init`, TanStack Query and Router, Lightweight Charts, Recharts, Motion, Lucide.
2. Put the colour, type and spacing tokens from the design table into `src/index.css` under `@theme`; load Inter and JetBrains Mono.
3. Build the shell: sidebar, top bar with live dot and horizon selector, footer disclaimer; routes `/`, `/coin/$pair`, `/replay`, `/honesty`, `/method`.
4. Connect Cloudflare Pages now so every merge to main gets a preview URL.

Why. Tokens first means every later screen is consistent without extra design time.

Done when. Shell renders on a phone and a laptop; preview URL live.

```
npm install @tanstack/react-query @tanstack/react-router @tanstack/react-table lightweight-charts \
  @tradingview/lwc-plugin-vertical-line recharts motion lucide-react sonner
npm install -D tailwindcss @tailwindcss/vite openapi-typescript @playwright/test
npx shadcn@latest add table card tabs badge button tooltip command select switch skeleton
```

F2 Radar and Markets (home page) · Aveek · Tue 09:00–13:00

Do

1. Two tabs. **Pump radar**: TanStack Table of scored coins — rank, coin, live price, risk % at the selected horizon with a level badge (Normal / Watch / Alert), tier, 2-hour sparkline, minutes since last alert. **All markets**: every Binance spot coin with live price, 24 h change and volume, prices ticking every 2 s; scored coins carry their risk badge, others "not scored".
2. Horizon tabs (1, 5, 10, 30, 60 min) stored in the URL; search box; tier filter; sort by risk by default.
3. Live updates: one `EventSource` writes `prices` and `tick` events into the query cache; rows re-rank with Motion.
4. Alerting the investor: a new alert raises a Sonner toast and, after the user clicks "Enable alerts", a browser notification (`Notification.requestPermission()`) that works while the tab is in the background.
5. Summary strip above the table: coins monitored, alerts in the last hour vs budget, last update time.

Why. This is the screen a user opens first; it must answer "what should I be careful with right now?" in one glance.

Done when. Prices tick every 2 s and scores every minute from the mock backend; a mock alert produces a toast and a notification.

```
export function useLiveTicks() {
  const qc = useQueryClient();
  useEffect(() => {
    const es = new EventSource(`${import.meta.env.VITE_API_URL}/stream`);
    es.addEventListener("tick", (e) => {
      const tick: RadarTick = JSON.parse((e as MessageEvent).data);
      qc.setQueryData(["radar"], tick.rows);
      tick.newAlerts.forEach((a) => toast.warning(`${a.pair}: pump risk ${Math.round(a.risk * 100)}%`));
    });
    return () => es.close();                      // the browser reconnects automatically on drops
  }, [qc]);
}
```

F3 Coin view · Aveek · Tue 14:00–18:00

Do

1. Lightweight Charts: candlesticks for the last 6 hours, the risk series as a line on a second price scale, and markers where alerts fired.
2. Drivers card: the five reasons for the current score in plain language, each with a small bar showing its size.
3. Header: coin, last price, 24 h volume, tier, current risk per horizon.

Why. An alert without a reason is not trusted; this screen carries the interpretability claim.

Done when. Opening any coin from the radar shows chart, risk line, markers and drivers.

F4 Replay · Aveek · Tue 18:00–22:00

Do

1. Event picker (shadcn Command) listing test-era events from `/replay/events`.
2. Play, pause and 1×/4×/16× speed; a time scrubber; a vertical line at the onset (vertical-line plugin).
3. Side panel: the pumped coin's rank among all coins each minute, and a result card — "first alert 7 min before onset, rank 2 of 61".
4. Animate client-side from the replay JSON; no server work during playback.
5. **Past-pump scoreboard** above the player: a table of every held-out pump — date, coin, minutes of warning before onset (or "missed"), rank at onset — with the share caught and the median lead at the top. Clicking a row replays it.

Why. Large pumps on Binance are now rare, so replay of held-out pumps is the reliable demo and the most convincing screen.

Done when. The scoreboard lists every test event, and any event replays smoothly with its lead time.

F5 Honesty and Method pages · Aveek · Wed 11:00–13:00

Do

1. Honesty: KPI tiles (precision, recall and median lead at the alert budget; Hit@5), a Recharts bar of alerts per hour vs budget, and coins monitored now vs the training-era median.
2. Method: the lead-time curve and inflation figures from the paper, with three plain-language paragraphs and a link to the PDF.

Why. Publishing its own error rate and method is what makes the platform credible rather than another "pump signal" site.

Done when. Both pages read from `/honesty` and static figures.

F6 Integration and polish · Aveek · Wed 14:00–18:00

Do

1. Switch the deployed API to the real model (after B4). Check one live coin's risk in the UI matches the engine log.
2. Loading skeletons, empty states, the feed-down banner; keyboard focus styles; check at 375 px width.
3. One Playwright test: open radar, open a coin, play a replay to the end.

Why. Polish is what makes a deployed site feel finished; the test catches a broken screen before the demo.

Done when. Lighthouse accessibility score above 90; Playwright test passes against the public URL.

F7 Backup video and README · Aveek · Wed 18:00–20:00

Do

1. Record a 2–3 minute screen capture following the demo script (Section 7).
2. README: what PumpRadar does, the live URL, screenshots, local quick start (`docker compose up` and `npm run dev`), data sources, ethics note.

Why. If the network fails on Thursday, the video still shows the system working; the README is what visitors read first.

Done when. Video link and README on main.

5D. Paper — all

W1 Paper skeleton · Hannah · Wed 10:00

Do. Overleaf project from the IEEE conference template; all section headings; figure placeholders with the exact filenames above; `refs.bib` with the papers in Section 10. Share edit access with the team.

Why. Placeholders force agreement on which exhibits exist.

Done when. PDF compiles with empty sections.

Section	Owner	Write on	Inputs
Introduction	Aveek	Wed 21:30–23:00	Contributions (2.2), headline numbers
Related work	Hannah	Wed 10:00–13:00	Novelty table (Section 10)
Data	Hannah	Wed 10:00–13:00	data_summary.json, onset histogram
Method	Hannah drafts, Kaashvi checks	Wed 15:30–19:00	Steps D3–D4, M1–M2, M6
Results	Kaashvi	Wed 17:00–22:00	metrics_A/B.json, all figures
System	Aveek	Wed 20:00–21:30	Architecture diagram, screens, latency
Discussion, limitations, abstract	All	Wed 21:00 – Thu 09:00	Everything

W2 Number check · Hannah · Wed 22:00

Do. List every number in the draft with its source file (results/*.json) and tick each one.

Why. One wrong number found by an examiner undermines all the others.

Done when. Every number ticked.

W3 Final read and export · All · Thu 08:00–10:00

Do. Each person reads one section they did not write; fix; export PDF; tag the repo `v1.0`.

Why. You do not see the gaps in your own writing.

Done when. Final PDF and tag.

Limitations to state honestly

- Events cover Binance, 2018–21; pump groups have since moved largely to smaller exchanges.
- Event lists are incomplete, so some "normal" coin-minutes are unrecorded pumps; measured precision is therefore conservative.
- Experiment A keeps only the 6 hours before each pump; Experiment B uses minute bars, not trade-level rush orders.
- Leave-coin-out and later-era tests are Future Work; so is scoring USDT pairs, which needs retraining on USDT data.
- Live risk scores update every minute (the model's resolution), while prices update every two seconds.
- Dual use: a published detector also shows manipulators what to avoid. The system uses public data only and places no trades.

6. Timeline: Monday 26 to Thursday 29 October

Plan for long days: about 7 hours on Monday evening and 12–13 hours on Tuesday and Wednesday; the platform is the tightest track. Bold rows are checkpoints; if a checkpoint output is missing, the next block starts by fixing it.

Time	Kaashvi	Hannah	Aveek
Mon 26 16:00–17:00	**All: kickoff — read Sections 1–4, G1–G6, agree contracts and API shapes**		
17:00–20:00	M1 Experiment A data	D1 events; start D2 downloads	B1 API skeleton, contract, mock mode
20:00–23:00	M2 splits and metrics + tests	D3 onset detection	F1 scaffold, design tokens, shell
23:00	**Sync: sapA.parquet ready; downloads running; mock API streaming**		
Tue 27 09:00–13:00	M3 Experiment A, all models	D4 features + xsec.parquet	F2 Radar
13:00	**Sync: first inflation numbers; xsec.parquet in release**		
14:00–18:00	M4 lead-time curve, canary, bootstrap	B2 live ingestor	F3 Coin view
18:00–22:00	M5 Experiment B starts	B3 scorer, alerts, SQLite; D5 leakage test	F4 Replay
22:00	**Checkpoint 1: Experiment A complete; engine scoring live with dummy model; Radar, Coin, Replay exist**		
Wed 28 09:00–13:00	M5 finish; M6 threshold + frozen model	D6 data summary; W1; Data and Related work	B5 deploy (mock mode); F5 Honesty and Method
13:00	**Checkpoint 2: model files to Hannah and Aveek**		
14:00–18:00	M7 interpretability; M8 freeze at 18:00	B4 replay files (13:30–15:30); Method draft	Redeploy with real model (15:30); F6 polish and test
18:00–23:00	Results section; check Method	Method draft; W2 number check (22:00)	F7 video, README; System; Introduction
23:00	**Checkpoint 3: full draft; code frozen; public URL running the real model**		
Thu 29 08:00–10:00	**All: W3 final read, export PDF, tag v1.0; rehearse the demo twice on the public URL**		
Demo	Presents results	Presents data and method	Runs the live demo

If you fall behind, cut in this order. Research: SVM with RBF kernel, KNN tuning, SHAP (keep permutation importance), the 1:5 comparison row. Platform: Playwright test, browser notifications, Motion animations, the Method page, the Coin view's drivers bars. **Never cut** Replay, the public deployment, the honest split, the leakage audit, or the results freeze.

7. Demo script (4 minutes)

Time	Who	Screen	What to say and show
0:00–0:40	Hannah	Title slide or README	The problem, the question "how early, honestly, and which coin", the data
0:40–1:30	Kaashvi	Figures	Inflation per lead time (prior protocol vs honest), lead-time curve, Hit@5
1:30–2:20	Aveek	Radar, Coin view	On the public URL: live coins ranked every minute; open one coin, show the risk curve and the five drivers
2:20–3:30	Aveek	Replay	Replay a test-era pump the model never saw; the alert fires; read out the lead time and rank
3:30–4:00	Kaashvi	Honesty	Alerts per hour vs budget; precision and recall at that budget; limitations in one line

Before the demo: start the app 30 minutes early so the 24-hour baseline is loaded; pre-select the replay event; keep the backup video open in a tab.

8. Risks and fallbacks

Risk	Early sign	Fallback
Daily kline files missing for 2018	Mon 21:00: most 2018 requests return 404	Switch those months to monthly files (D2 step 4)
Downloads too slow	Tue 08:00: under 70% done	Run Experiment B on events with complete data; report coverage
Sapienza file layout differs	M1 column check fails	Recompute the 25 s chunks from their downloader.py trades for 100 pumps
Memory errors	Training crashes	Subsample negatives with weights; use LightGBM; run on Kaggle or Colab
Weak signal beyond 10 minutes	PR-AUC near the base rate	This is a finding: report the warning horizon with intervals
Binance blocks the VM region	Wed 09:00: ping returns 451 or 403	Recreate the VM in another Indian or Singapore region; Replay and Honesty work without Binance
Binance unreachable during the demo	Feed-down banner on the site	Present Replay and Honesty (stored data); play the backup video
Frontend behind schedule	Wed 13:00: a page unfinished	Apply the platform cut list (Section 6); Radar and Replay first
Front and back disagree on data shapes	Type errors after regenerating types	Change schemas.py first, regenerate types.ts, fix both in one pull request
CORS or HTTPS errors after deploy	Browser console errors	WEB_ORIGIN must equal the Pages URL exactly; check Caddy logs for the certificate
Feature mismatch between site and research	Replay lead differs from offline	Engine imports features.py; feature order read from features.json
Someone is blocked	Issue idle for an hour	Label it blocker, message the owner, move to the next issue

9. Algorithms, metrics and terms

Algorithm	Role
Logistic regression	Main interpretable model; coefficients show which signals warn and how early
K-nearest neighbours	Course baseline; trained on a balanced subsample
Decision tree (depth ≤ 6)	Readable alert rules
Random forest	Best model in prior work; direct comparison
Support vector machine (linear)	Course baseline
LightGBM	Strong reference; likely deployed model
Rush-order threshold / volume z-score rank	Simple baselines every model must beat

Term	Meaning
Leakage	A model receives information in training it would not have in real use, inflating test scores
Random vs honest split	Rows shuffled at random, versus whole pumps kept together with past training and future testing
Inflation	Score under the random split minus score under the honest split
Onset	The minute buying actually explodes on the exchange (can precede the Telegram post)
Lead time / horizon	Minutes of warning before onset / the window asked about, e.g. within 10 minutes
Cross-sectional	Comparing all coins at the same minute; the pumped coin is the positive, the rest negatives
Base rate	Odds of being right by chance; about 1 in 60 coins per event in Experiment B
PR-AUC	Summary of precision and recall over all thresholds; the right score when positives are rare
Hit@5	Share of pumps where the pumped coin ranks in the model's top 5
Alert budget	Promised maximum false alarms per hour; sets the alert threshold
Robust z-score	Distance from a coin's own median in units of its normal spread (IQR ÷ 1.349)
Quiet accumulation	Volume rising while price barely moves: insiders buying unnoticed
Event-level bootstrap	Resampling whole pumps to give honest confidence intervals

10. Prior work and sources

Work	What it did	What this project adds
La Morgia et al., ICCCN 2020; ACM TOIT 2023	Rush-order detection within 25 s of onset; public Binance dataset	Warning before onset; honest split
Fantazzini and Xiao, Econometrics 2023	Flags the 60 min before announcement; random forest AUC ≈ 0.99 under 5-fold CV over chunks	Inflation measured; minute-level curve; all coins
Xu and Livshits, USENIX Security 2019	Predicts the target coin about 1 h ahead from hourly data	Minute resolution with microstructure features
Hu et al., SIGMOD 2023	Target coin from Telegram channel history	Market data only
Gogol et al., 2024	Top-5 of 50 coins: 56% at 20 s, 19% at 1 min before (43 events)	Hundreds of events, curve with intervals
Karbalaii, 2025	About 70% of pre-event volume trades within 1 h of announcement	Tested as quiet-accumulation features

- SystemsLab-Sapienza pump-and-dump dataset — github.com/SystemsLab-Sapienza/pump-and-dump-dataset
- Fantazzini and Xiao (2023) — mpra.ub.uni-muenchen.de/118435
- La Morgia et al. (2020) — arxiv.org/abs/2005.06610; (2023) — arxiv.org/abs/2105.00733
- Xu and Livshits (2019) — usenix.org/conference/usenixsecurity19/presentation/xu-jiahua
- Hu et al. (2023) — arxiv.org/abs/2204.12929
- Gogol et al. (2024) — arxiv.org/abs/2412.18848
- Karbalaii (2025) — arxiv.org/abs/2504.15790
- Binance public data archive — data.binance.vision; market-data-only API — developers.binance.com/docs/binance-spot-api-docs/faqs/market_data_only